

Testo e Soluzioni Esame Programmazione di Sistema

Data: 2024-07-15

Domanda 1

Testo: Si considerino le seguenti strutture dati e rispettive porzioni di codice. Per ciascuna di esse si indichi la dimensione di memoria allocata nello stack e nello heap.

Listing 1: Struttura 1

```
1 use std::rc::Rc;
2 let rc: Rc<u64> = Rc::new(42);
3 let rc2 = rc.clone();
4 let wk = Rc::downgrade(&rc);
```

Listing 2: Struttura 2 e codice di supporto

```
1 let mut vector = Vec::<u64>::with_capacity(8);
2 for i in 0..5 {
3     vector.push(i);
4 }
5 let vslice = &vector[1..3];
```

Soluzione: Di seguito riportiamo uno schema visuale dell'allocazione in memoria (Stack e Heap) e un'analisi profonda di ogni singolo campo.

Analisi Struttura 1: Rc<u64>

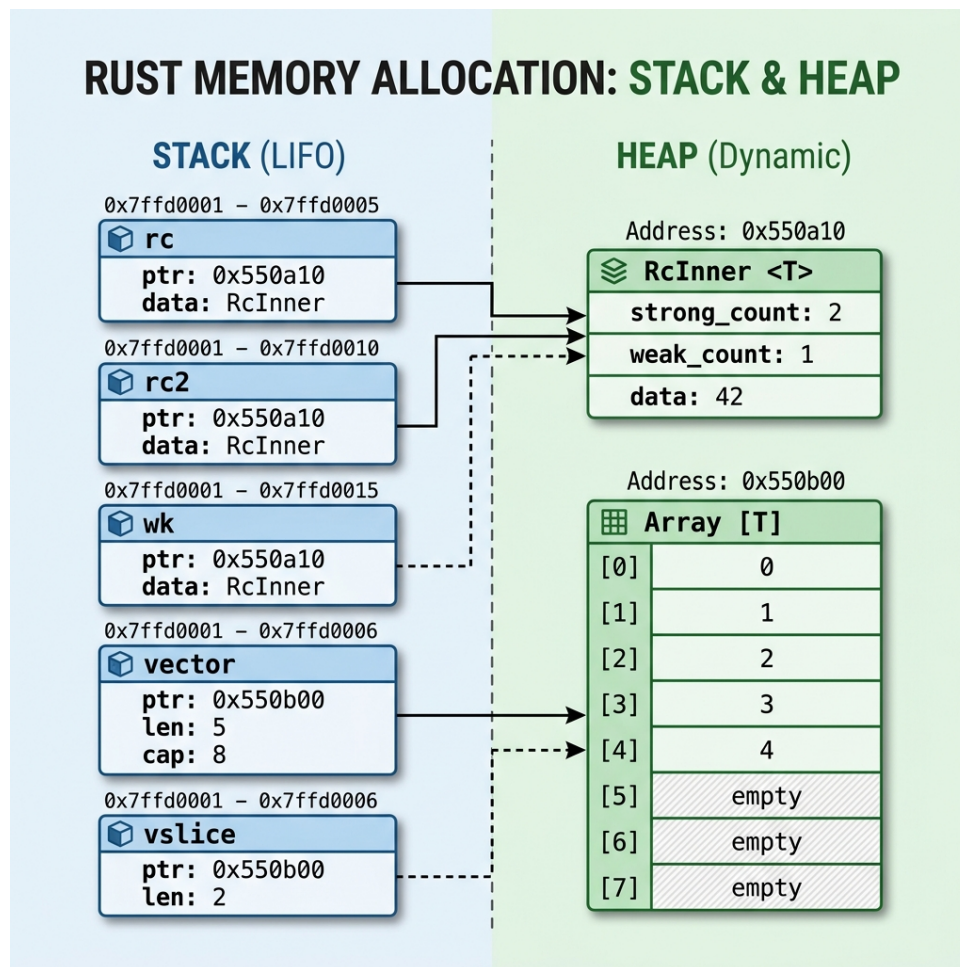
Il tipo Rc<T> (Reference Counted) è uno smart pointer che consente la proprietà condivisa di un dato allocato sullo heap in ambienti single-thread.

Allocazione sullo Stack:

- rc, rc2, e wk sono variabili allocate sullo Stack.
- Ognuna di queste variabili è fisicamente un singolo puntatore (un "thin pointer") di 8 byte (su architetture a 64-bit).
- Questo puntatore punta a un blocco contiguo allocato sullo Heap, chiamato internamente RcBox o RcInner.
- **Perché serve?** Il puntatore sullo stack è necessario per permettere al programma di riferirsi alla memoria condivisa e tenerne traccia in modo performante durante lo scope della funzione, fornendo accesso istantaneo.

Allocazione sullo Heap: Un singolo blocco RcBox viene allocato sullo Heap durante la chiamata a Rc::new(42). Questo blocco condiviso è composto da tre campi:

- **strong_counter** (8 byte): Tiene traccia del numero di Rc attivi (nel nostro caso 2, rc e rc2). Quando questo contatore arriva a zero, il dato effettivo (**data**) viene deallocato.



2. **capacity** (8 byte): Indica quanti elementi possono essere contenuti nell'allocazione heap prima di dover effettuare una riallocazione (valore: 8). Protegge contro continue richieste lente di espansione della memoria al sistema operativo.
 3. **length** (8 byte): Indica quanti elementi effettivi sono attualmente inizializzati nel vettore (valore: 5). Questo evita letture di memoria non valida (garbage bytes) oltre i limiti conosciuti.
- **vslice** (16 byte): È uno "slice" preso in prestito. Essendo una "view" dinamica su una parte della memoria, è un fat pointer composto da:
 1. **ptr** (8 byte): Un puntatore al primo elemento rilevante dello slice (cioè all'indirizzo che corrisponde all'indice 1 del vettore originale).
 2. **length** (8 byte): Il numero di elementi racchiusi nello slice (nel nostro caso, l'intervallo 1..3 contiene esattamente 2 elementi). Serve al compilatore per effettuare il "bounds checking" ed evitare accessi ad elementi fuori dallo slice.

Allocazione sullo Heap: Sullo heap viene allocato un singolo array contiguo al momento dell'invocazione di `Vec::with_capacity(8)`.

- **Array Raw** (64 byte totali): È un blocco di memoria in grado di contenere 8 elementi di tipo `u64` (8 elementi x 8 byte ciascuno = 64 byte).
- **Perché serve?** Il vettore per sua natura può crescere e restringersi a runtime. Poiché tutte le variabili sullo stack devono per forza avere una dimensione predeterminata, nota e immutabile a compile-time, i dati strutturati dinamicamente devono essere obbligatoriamente scaricati sullo heap, dove l'allocatore può ritagliare o espandere sequenze di byte su richiesta.

Domanda 2

Testo: Si consideri il programma seguente. Spiegare i problemi di questo codice e come risolverli.

```
1 use std::sync::{Arc, Mutex, Condvar};
2 use std::thread;
3
4 fn main() {
5     let data = Arc::new((Mutex::new(Vec::new()), Condvar::new()));
6
7     let data_clone1 = Arc::clone(&data);
8     let data_clone2 = Arc::clone(&data);
9
10    let writer = thread::spawn(move || {
11        let (lock, cvar) = &*data_clone1;
12        for i in 1..=5 {
13            thread::sleep(std::time::Duration::from_secs(1));
14            let mut data = lock.lock().unwrap();
15            data.push(i);
16            cvar.notify_all();
17        }
18    });
19
20    let reader1 = thread::spawn(move || {
21        let (lock, cvar) = &*data_clone2;
22        loop {
23            let mut data = lock.lock().unwrap();
24            data = cvar.wait(data).unwrap();
25            if let Some(value) = data.pop() {
26                println!("Reader 1: read {}", value);
27            } else {
28                println!("{}", (1));
29            }
30        }
31    });
32
33    let reader2 = thread::spawn(move || {
34        let (lock, cvar) = &*data;
35        loop {
36            let mut data = lock.lock().unwrap();
37            data = cvar.wait(data).unwrap();
38            if let Some(value) = data.pop() {
39                println!("Reader 2: read {}", value);
40            } else {
41                println!("{}", (2));
42            }
43        }
44    });
45
46    writer.join().unwrap();
47    reader1.join().unwrap();
48    reader2.join().unwrap();
49 }
```

Soluzione (Problemi del Codice): Il programma presentato soffre di un caso classico di **Deadlock** causato da un loop infinito asimmetrico. Il thread **writer** esegue un loop finito di 5 iterazioni, durante le quali inserisce elementi e chiama **notify_all()**. Tuttavia, i due thread **reader** possiedono un loop infinito senza una condizione di uscita sicura.

Una volta che i 5 elementi sono stati consumati in sequenza dai reader, entrambi rimarranno in un **wait()** perenne sulla condition variable, poiché non c'è più alcun writer vivo in grado di

inviare ulteriori `notify_all()`. Questo impedisce ai thread di terminare e causa il blocco totale del programma (anche il `main` non uscirà mai, bloccato sulle `join`). Inoltre, la logica attuale genera problemi legati agli *spurious wakeups*.

Soluzione (Approccio Flag): Per risolvere, inseriamo un booleano all'interno del `Mutex` che indichi se la fase di scrittura è terminata. I reader devono verificare questa condizione al risveglio, ed effettuare un `break` se il vettore è vuoto e la scrittura è conclusa.

```
1 // Modifica nel main
2 let data = Arc::new((Mutex::new((Vec::new(), false)), Condvar::new()));
3
4 // Modifica nel Writer (alla fine delle 5 iterazioni):
5 let mut lock_data = lock.lock().unwrap();
6 lock_data.1 = true;
7 cvar.notify_all();
8
9 // Modifica nei Reader (all'interno del loop):
10 let mut lock_data = lock.lock().unwrap();
11 while lock_data.0.is_empty() && !lock_data.1 {
12     lock_data = cvar.wait(lock_data).unwrap();
13 }
14
15 if let Some(value) = lock_data.0.pop() {
16     println!("Reader: read {}", value);
17 } else if lock_data.1 {
18     // Il Writer ha finito ed il vettore e' vuoto: usciamo
19     break;
20 }
```

Domanda 3

Testo: Si implementi in RUST la struct `CountDownLatch` che permette a uno o più thread di attendere che un gruppo di operazioni eseguite da altri thread siano eseguite. Essa incapsula un contatore ed offre i seguenti tre metodi thread-safe (oltre alla propria funzione costruttrice) che devono essere implementati:

```
1 fn new(n: usize) -> Self
2 // inizializza la struttura, impostando ad n il contatore
3
4 fn count_down(&self)
5 // decrementa il contatore, se maggiore di 0
6
7 fn wait(&self)
8 // blocca l'esecuzione del chiamante senza consumare cicli di cpu, finche' il
  contatore non diventa zero
9
10 fn wait_timeout(&self, d: Duration) -> std::sync::WaitTimeoutResult
11 // blocca l'esecuzione del chiamante senza consumare cicli di cpu, in attesa
   che il contatore raggiunga 0 per una durata massima pari a d, restituendo il
   risultato dell'attesa.
```

Soluzione: L'implementazione sfrutta un `Mutex` per proteggere un contatore interno e un `Condvar` per gestire l'attesa asincrona. **Nota architetturale:** è idiomatico in Rust omettere `Arc` dall'implementazione interna di queste primitive di sincronizzazione, demandando all'utente del main la scelta del modello di condivisione della memoria (tramite `Arc` o `Scoped Threads`).

```
1 use std::sync::{Mutex, Condvar, WaitTimeoutResult};
2 use std::time::Duration;
3
4 pub struct CountDownLatch {
5     state: Mutex<usize>,
6     cvar: Condvar,
7 }
8
9 impl CountDownLatch {
10     pub fn new(n: usize) -> Self {
11         CountDownLatch {
12             state: Mutex::new(n),
13             cvar: Condvar::new(),
14         }
15     }
16
17     pub fn count_down(&self) {
18         let mut count = self.state.lock().unwrap();
19         if *count > 0 {
20             *count -= 1;
21             if *count == 0 {
22                 // Notifica tutti i thread in attesa quando arriva a zero
23                 self.cvar.notify_all();
24             }
25         }
26     }
27
28     pub fn wait(&self) {
29         let mut count = self.state.lock().unwrap();
30         // Ciclo while nativo di rust per gestire i wakeup spuri
31         let _count = self.cvar.wait_while(count, |c| *c > 0).unwrap();
32     }
33
34     pub fn wait_timeout(&self, d: Duration) -> WaitTimeoutResult {
35         let count = self.state.lock().unwrap();
```

```
36         let (_count, timeout_result) = self.cvar.wait_timeout_while(count, d, |  
    c| *c > 0).unwrap();  
37         timeout_result  
38     }  
39 }
```

Domanda 4

Testo: Si descriva nel dettaglio il comportamento di questo programma. Si spieghi l'errore di compilazione che viene generato. Che cosa deve essere modificato per permetterne la compilazione in modo da ottenere la visualizzazione di "Numero: 4"?

```
1 fn main() {
2     let mut data = vec![1, 2, 3, 4, 5];
3     data.push(60);
4
5     let mut process_data = move || {
6         data.push(50);
7         let count = data.iter().filter(|&x| x % 2 == 0).count();
8         println!("Numero: {:?}", count);
9     };
10
11     data.push(40);
12     process_data();
13     data.push(30);
14 }
```

Soluzione:

Errore di Compilazione: Il programma definisce una closure che utilizza la keyword `move`. Questa sintassi trasferisce in modo permanente la proprietà (ownership) della variabile originale `data` all'interno della closure stessa. Di conseguenza, alla riga 10, quando il codice cerca di invocare `data.push(40)`, il compilatore Rust interviene bloccando l'operazione con un errore fatale del tipo "use of moved value". Non è matematicamente possibile accedere a una risorsa che è stata già ceduta definitivamente a un altro scope.

Soluzione Riparativa: Per risolverlo, bisogna clonare il vettore un istante prima della definizione della closure. Siccome l'obiettivo dell'esercizio richiede che `process_data()` stampi esattamente l'output "Numero: 4" (e contando che aggiungerà internamente il numero 50 pari), dobbiamo assicurarci che il vettore, al momento della clonazione, possieda un totale di 3 numeri pari pre-esistenti. Questo accade perfettamente riga 3! Il vettore originale `[1, 2, 3, 4, 5]` più l'aggiunta di 60 genera un vettore con i numeri pari 2, 4, e 60.

```
1 fn main() {
2     let mut data = vec![1, 2, 3, 4, 5];
3     data.push(60);
4
5     // Effettuiamo un clone del dato e diamo in pasto QUESTO alla closure!
6     let mut data_clone = data.clone();
7
8     let mut process_data = move || {
9         data_clone.push(50);
10        let count = data_clone.iter().filter(|&x| x % 2 == 0).count();
11        println!("Numero: {:?}", count);
12    };
13
14    // Ora data e' libero di essere mutato indipendentemente
15    data.push(40);
16    process_data();
17    data.push(30);
18 }
```